Table des matières

Labo 01 — Voir l'architecture à l'œuvre 1
Prérequis 1
Étape 1 — Fixer le contexte, et voir la précédence des paramètres 1
Étape 2 — Stockage et calcul découplés 2
Étape 3 — Le cache de résultats (couche services cloud) 3
Étape 4 — Le pruning de micro-partitions (dans le Query Profile) 3
Étape 5 — Les types de tables 4
Étape 6 — Une colonne semi-structurée (structuré + VARIANT ensemble) 5
Étape 7 — Les types de vues 5
Étape 8 — La hiérarchie, d'un coup d'œil 6
Ménage (optionnel) 6
Récapitulatif 6

Labo 01 — Voir l'architecture à l'œuvre

Module 1/8 — Formation Snowflake (préparation SnowPro Core, COF-C03)

Ce que fait ce labo — Vous vérifiez de vos yeux les idées du cours : le découplage stockage/calcul, le cache de résultats, le pruning de micro-partitions (dans le *Query Profile*), les types de tables, une colonne semi-structurée, et les types de vues. Comptez 25 à 35 minutes. Tout se fait dans un worksheet Snowsight.

Prérequis

- Le Labo 00 est fait : l'entrepôt `SNOWPRO_WH` et la base `SNOWPRO_DB.SANDBOX` existent.

Étape 1 — Fixer le contexte, et voir la précédence des paramètres

Théorie : `Theorie_01_SNOW.md`, section 8.

```
-- Les « 3 questions » : rôle, entrepôt, base/schéma.
USE ROLE SYSADMIN;
```


© 2026 iAlexMG. Tous droits réservés.

```sql
USE WAREHOUSE SNOWPRO_WH;
USE DATABASE SNOWPRO_DB;
USE SCHEMA SANDBOX;

-- La valeur en vigueur d'un paramètre (ici, le délai maximal d'une requête).
SHOW PARAMETERS LIKE 'STATEMENT_TIMEOUT_IN_SECONDS';   -- regardez la colonne "level"

-- On l'écrase au niveau SESSION...
ALTER SESSION SET STATEMENT_TIMEOUT_IN_SECONDS = 3600;
SHOW PARAMETERS LIKE 'STATEMENT_TIMEOUT_IN_SECONDS';   -- "level" = SESSION : la session l'emporte
```

La colonne « level » — `SHOW PARAMETERS` n'affiche pas un texte, mais une grille dans le panneau Results (en bas du worksheet). Ses colonnes : `key` (le nom du paramètre), `value` (la valeur en vigueur), `default` (la valeur d'usine), `level` (le niveau où la valeur a été fixée), `description` et `type`. Si vous ne voyez pas `level`, faites défiler la grille vers la droite (les colonnes continuent au-delà du bord visible) ou élargis le panneau. Lecture de `level` : vide = personne n'y a touché (valeur par défaut) ; `SESSION` = fixée pour votre session (elle l'emporte) ; vous pourriez aussi voir `ACCOUNT` ou `USER`. Enfin, pour les voir tous, lancez `SHOW PARAMETERS;` sans `LIKE` : le `LIKE '…'` ne fait que filtrer sur un nom.

À retenir : la valeur fixée pour la session gagne sur celle de l'utilisateur, qui gagne sur celle du compte (compte < utilisateur < session).

Étape 2 — Stockage et calcul découplés

Théorie : `Theorie_01_SNOW.md`, sections 1 et 5.

Deux entrepôts indépendants lisent la même donnée, sans se gêner :

```sql
-- Un second moteur, tout aussi petit et économe.
CREATE WAREHOUSE IF NOT EXISTS SNOWPRO_WH2
  WAREHOUSE_SIZE='XSMALL' AUTO_SUSPEND=60 AUTO_RESUME=TRUE INITIALLY_SUSPENDED=TRUE;

-- Le premier moteur lit les commandes d'exemple...
USE WAREHOUSE SNOWPRO_WH;
SELECT COUNT(*) FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS;

-- ...le second lit EXACTEMENT la même table, indépendamment.
USE WAREHOUSE SNOWPRO_WH2;
SELECT COUNT(*) FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS;
```

À quoi s'attendre : ~1 500 000 lignes, deux fois. Les données ne sont pas copiées d'un entrepôt à l'autre : il y a une seule copie dans le stockage, lue par deux calculs séparés. C'est le découplage en action.

© 2026 iAlexMG. Tous droits réservés.

La table `SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS` — On ne l'a pas « sélectionnée » avec un `USE` : on l'a désignée par son chemin complet à trois niveaux, `BASE.SCHÉMA.OBJET`, soit la base `SNOWFLAKE_SAMPLE_DATA`, le schéma `TPCH_SF1`, la table `ORDERS`. Un nom ainsi qualifié l'emporte sur le contexte de l'Étape 1 : votre `USE SCHEMA SANDBOX` reste actif, la requête va simplement lire ailleurs le temps de s'exécuter. Cette base est livrée d'office par Snowflake, en lecture seule, via le partage de données, d'où sa présence sans que vous l'ayez créée. Et `TPCH_SF1` désigne le jeu TPC-H à l'échelle SF1 (*Scale Factor 1*, ~1,5 M commandes) : assez gros pour les démos, assez petit pour rester gratuit.

Étape 3 — Le cache de résultats (couche services cloud)

Théorie : `Theorie_01_SNOW.md`, section 1.

```
USE WAREHOUSE SNOWPRO_WH;

-- 1re exécution : l'entrepôt calcule (quelques secondes).
SELECT O_ORDERSTATUS, COUNT(*) AS n, AVG(O_TOTALPRICE) AS panier_moyen
FROM   SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS
GROUP  BY O_ORDERSTATUS;

-- Relancez la MÊME requête, sans rien changer : réponse quasi instantanée.
```

À quoi s'attendre : la 2e exécution revient en un éclair. Snowflake ressert le cache de résultats (géré par les services cloud) sans rallumer l'entrepôt. Ouvrez Query History (menu *Monitoring*) : la 2e requête affiche une durée quasi nulle et « *query result reuse* ».

Étape 4 — Le pruning de micro-partitions (dans le Query Profile)

Théorie : `Theorie_01_SNOW.md`, section 6.

Au retour d'un *sign out* — Le contexte de session (rôle, entrepôt, base) est reparti à zéro : rejouez d'abord l'Étape 1. Sinon la requête peut tourner sans entrepôt actif, et Snowsight affiche « *Query Details unavailable* » à la place du profile.

Le pruning ne mord que si la colonne filtrée est corrélée au rangement physique des micro-partitions. On le prouve par un contraste : la même donnée, d'abord non rangée, puis triée par date.

```
-- 1) La table d'exemple n'est PAS rangée par date : chaque micro-partition couvre ~1992→1998,
--    donc ses bornes min/max recoupent toujours la plage et aucune ne peut être sautée.
SELECT COUNT(*)
FROM   SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS
WHERE  O_ORDERDATE BETWEEN '1995-01-01' AND '1995-03-31';    -- attendu : scanned = total
```


© 2026 iAlexMG. Tous droits réservés.

```sql
-- 2) On recopie la table en la TRIANT physiquement par date...
CREATE OR REPLACE TABLE SANDBOX.orders_by_date AS
SELECT * FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS
ORDER  BY O_ORDERDATE;

-- 3) ...et le MÊME filtre élague enfin pour de vrai.
SELECT COUNT(*)
FROM   SANDBOX.orders_by_date
WHERE  O_ORDERDATE BETWEEN '1995-01-01' AND '1995-03-31';   -- attendu : scanned << total
```

Puis, dans Snowsight : Query History → cliquez la requête n° 3 → onglet *Query Profile*. Sur le nœud `TableScan`, comparez « Partitions scanned » et « Partitions total ».

À quoi s'attendre : sur la requête 1, *scanned* = *total*, il n'y a rien à élaguer. Sur la requête 3, *scanned* est bien plus petit que *total*. La seule chose qui a changé, c'est l'ordre physique des données : le `ORDER BY` du chargement a regroupé les dates voisines dans les mêmes micro-partitions, leurs bornes min/max se séparent, et Snowflake peut sauter tout ce qui est hors plage. Personne n'a créé d'index, mais le pruning dépend du rangement, il n'est pas automatique sur n'importe quelle colonne.

Si le Query Profile reste indisponible (requête trop courte, ou chemin optimisé qui n'émet pas de profile), lisez les mêmes chiffres directement en SQL : `GET_QUERY_OPERATOR_STATS` est le Query Profile servi en table. Lancez ce bloc juste après la requête 3 :

```sql
SELECT operator_type,
       operator_statistics:pruning:partitions_scanned::int AS partitions_scanned,
       operator_statistics:pruning:partitions_total::int   AS partitions_total
FROM   TABLE(GET_QUERY_OPERATOR_STATS(LAST_QUERY_ID()))
WHERE  operator_type = 'TableScan';
```

Étape 5 — Les types de tables

Théorie : `Theorie_01_SNOW.md`, section 7.

```sql
CREATE OR REPLACE TABLE           SANDBOX.t_perm      (x INT);   -- permanente (défaut)
CREATE OR REPLACE TRANSIENT TABLE SANDBOX.t_transient (x INT);   -- pas de Fail-safe
CREATE OR REPLACE TEMPORARY TABLE SANDBOX.t_temp      (x INT);   -- le temps de la session

SHOW TABLES IN SCHEMA SANDBOX;
```

À quoi s'attendre : dans le résultat de `SHOW TABLES`, comparez les colonnes `kind` et `is_transient`. `t_temp` disparaîtra à la fin de votre session (rouvrez un worksheet plus tard : elle ne sera plus là), d'où son usage « jetable ».

© 2026 iAlexMG. Tous droits réservés.

Étape 6 — Une colonne semi-structurée (structuré + `VARIANT` ensemble)

Théorie : `Theorie_01_SNOW.md`, section 7.

```sql
CREATE OR REPLACE TABLE SANDBOX.evenements (
  id     INT,          -- structuré
  charge VARIANT       -- semi-structuré (du JSON tel quel)
);

INSERT INTO SANDBOX.evenements
  SELECT 1, PARSE_JSON('{"type":"clic","page":"accueil","tags":["a","b"]}');

-- On extrait des champs du JSON avec « : » et « [ ] ».
SELECT id,
       charge:type::string    AS type,
       charge:page::string    AS page,
       charge:tags[0]::string AS premier_tag
FROM   SANDBOX.evenements;
```

À quoi s'attendre : `clic | accueil | a`. La même table mêle une colonne structurée (`id`) et une colonne semi-structurée (`charge`), la preuve que « type de données » et « type de table » sont deux axes indépendants.

Étape 7 — Les types de vues

Théorie : `Theorie_01_SNOW.md`, section 7.

```sql
-- Vue standard : une requête enregistrée, ne stocke rien.
CREATE OR REPLACE VIEW SANDBOX.v_orders AS
  SELECT O_ORDERKEY, O_TOTALPRICE
  FROM   SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS
  SAMPLE (100 ROWS);

-- Vue sécurisée : même idée, mais sa définition est masquée (utile pour le partage).
CREATE OR REPLACE SECURE VIEW SANDBOX.v_orders_secure AS
  SELECT O_ORDERKEY
  FROM   SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS
  SAMPLE (100 ROWS);
```


© 2026 iAlexMG. Tous droits réservés.

```sql
-- La définition d'une vue standard est visible...
SELECT GET_DDL('VIEW', 'SANDBOX.v_orders');
```

À retenir : une vue standard recalcule à chaque lecture ; une vue matérialisée (Enterprise+) stockerait le résultat ; une vue sécurisée cache sa définition et bride certaines optimisations qui pourraient fuiter des données.

Étape 8 — La hiérarchie, d'un coup d'œil

Théorie : `Theorie_01_SNOW.md`, section 4.

```sql
SHOW WAREHOUSES;                    -- branche « calcul »
SHOW DATABASES;                     -- branche « données »
SHOW SCHEMAS   IN DATABASE SNOWPRO_DB;
SHOW TABLES    IN SCHEMA   SNOWPRO_DB.SANDBOX;

-- Nom complet = BASE.SCHÉMA.OBJET
SELECT * FROM SNOWPRO_DB.SANDBOX.evenements;
```

Ménage (optionnel)

```sql
DROP WAREHOUSE IF EXISTS SNOWPRO_WH2;   -- on garde SNOWPRO_WH pour la suite
DROP TABLE     IF EXISTS SANDBOX.orders_by_date;     -- la copie triée de l'Étape 4 (~1,5 M lignes)
ALTER SESSION UNSET STATEMENT_TIMEOUT_IN_SECONDS;    -- on relâche le paramètre forcé à l'Étape 1
-- t_temp part seule à la fin de session ; les autres objets sont minuscules et peuvent rester.
```

Récapitulatif

- ☐ J'ai vu deux entrepôts lire la même donnée (découplage).
- ☐ J'ai constaté le cache de résultats (2e exécution instantanée).
- ☐ J'ai trouvé « Partitions scanned / total » dans un Query Profile (pruning).
- ☐ J'ai créé une table permanente, transient et temporaire, et lu leur `kind`.
- ☐ J'ai mêlé structuré et `VARIANT` dans une seule table.

© 2026 iAlexMG. Tous droits réservés.

- ☐ Je distingue vue standard, matérialisée et sécurisée.

Tout coché, vous tenez le domaine le plus lourd de l’examen. Cap sur le Cours 02, les fonctionnalités IA/ML et le développement (Snowpark, Cortex…).

© 2026 iAlexMG. Tous droits réservés.